Artificial Intelligence 358 (2026) 104575

Contents lists available at ScienceDirect

Artificial Intelligence

journal homepage: www.elsevier.com/locate/artint

# Identifying heterophilic neighbors via confidence-based subgraph matching for graph neural networks

Yoonhyuk Choi [a,*], Chong-Kwon Kim [b,*]

[a] Sookmyung Women's University, Seoul, 04310, South Korea
[b] Korea Institute of Energy Technology, Naju, 58330, South Korea

ARTICLE INFO

*PACS:*
0000
1111

*2000 MSC:*
0000
1111

*Keywords:*
Graph neural networks
Graph representation learning
Graph similarity computation
Label propagation

ABSTRACT

Graph Neural Networks (GNNs) often struggle with heterophilic graphs, where neighboring nodes tend to have dissimilar labels-a common scenario in real-world networks. This paper addresses this limitation through a two-phase framework called ConSM (Confidence-based Subgraph Matching). First, we introduce a confidence-aware subgraph matching module that estimates edge coefficients by comparing the structural similarity of 2-hop neighborhoods using optimal transport. This process identifies task-irrelevant or misleading edges based on a tunable confidence ratio. Second, we integrate these edge coefficients into a sign-aware label propagation mechanism that adaptively encourages or discourages message passing based on edge confidence, thereby enhancing GNN robustness under heterophily. Compared to our earlier conference version [1], this manuscript provides (i) a clearer justification of key design choices such as subgraph-based reasoning and the use of 2-hop neighborhoods, (ii) an adaptive strategy to tune the confidence ratio without manual search, and (iii) extensive new experiments covering recent heterophily-oriented baselines and larger, leakage-free datasets. Empirical results show that ConSM improves classification accuracy, mitigates over-smoothing, and remains effective across both homophilic and heterophilic regimes.

## 1. Introduction

The investigation of graph-structured data has gained significant attention in various fields, including physics [2], protein-protein interactions [3], and social networks [4]. When integrated with deep neural networks (DNNs) [5], graph neural networks (GNNs) have achieved state-of-the-art performance by concurrently modeling node features and network structures [6–10]. Specifically, the message passing mechanism plays a crucial role by aggregating features from neighboring nodes [2]. Consequently, GNNs have consistently demonstrated superior performance in various tasks, including semi-supervised node classification and link prediction.

However, recent studies reveal that GNNs derive advantages from message passing under limited conditions, such as high assortativity of subject networks [11]. In this paper, we categorize networks into two types: homophilic (assortative) ones, where most edges connect two nodes with the same label, and heterophilic graphs, where most connections are disassortative. Most prior work on GNNs operates under the assumption that connected nodes are likely to possess the same label. This assumption leads to suboptimal performance on many real-world heterophilic datasets [12], which are characterized by connections between nodes with different labels.

* Corresponding authors.
*E-mail address:* chldbsgur123@gmail.com (Y. Choi).

https://doi.org/10.1016/j.artint.2026.104575
Received 31 July 2024; Received in revised form 14 March 2026; Accepted 31 May 2026
Available online 3 June 2026


**Fig. 1.** Node classification accuracy (%) of GCN on different datasets; (a) Cora, and (b) Chameleon. For each graph, we randomly prune a certain proportion of assortative/disassortative edges and plot their performance. We also describe a special case of (c) helpful aggregation scenario under disassortative graphs.

To address this challenge, many innovative schemes have been introduced. Some methods specify different weights for each connection [10,13–15] or remove disassortative edges altogether [16–18]. Other approaches leverage distant nodes with similar features [19–21] or apply different aggregation boundaries based on the central nodes [22]. Despite these advancements, there remains a critical question: *is it necessary to specify different weights for GNNs?*

To answer the above question, we conduct an investigation using two representative datasets: one is an assortative citation network called Cora [23], and the other is Chameleon [24], which contains many disassortative links between Wikipedia web pages. In Fig. 1, we randomly prune a specified fraction of assortative and disassortative edges and report the node classification accuracy of GCN [8]. Through this study, we observe two key characteristics: (1) For Cora, performance increases as assortative edges are maintained while disassortative edges are removed. In contrast, Chameleon data is heterophilic. Thus, disassortative links play an important role as the number of remaining assortative edges decreases [1]. We refer to Fig. 1-c to analyze this result. Although the graph is heterophilic, two central nodes share the same types of neighborhoods (2 green, 1 yellow), which can help distinguish them from others [25,26]. (2) The removal of assortative edges has a greater impact on overall performance than disassortative ones. For instance, the performance in Cora using the original graph is 79.8 % (top right). If we remove all disassortative edges, it attains 88.1 % (top left), whereas eliminating assortative links reduces performance to 51.1 % (bottom right).

We conjecture that removing a small proportion of assortative edges can be harmful, and thus, assigning accurate weights is fundamental for GNNs. Now, the problem is: *how can we determine these coefficients correctly and use them?* To achieve this, we focus on the GAM [27], which suggests a supplementary module with label propagation. Specifically, the supplementary module of GAM only utilizes a central node to mitigate noise. However, referring to Fig. 1, excluding all links of assortative and disassortative types shows the lowest performance, similar to GAM's method. To overcome this limitation, our supplementary module focuses on the widely used optimal transport [28–31] to measure similarity between two subgraphs. Additionally, we apply a confidence ratio to address multiple disassortative links. Considering these predictions as supplementary edge coefficients, we apply label propagation [32] to a certain proportion of high-confidence edges, while treating the remaining edges as disassortative to prevent the connected nodes from becoming similar. Our contributions can be summarized as follows:

- We introduce a confidence-based subgraph matching approach to retrieve edge coefficients accurately. Our model is scalable and generalizes well to both homophilic and heterophilic graphs by varying the confidence ratio.
- Assuming that a certain proportion of edges are disassortative, we enhance label propagation to prevent two nodes with a lower similarity score from becoming closer. Specifically, we divide the edge coefficients into two parts, guiding the positive pairs to be similar and the negative pairs to be dissimilar.
- We conduct extensive experiments on publicly available datasets to validate our approach. The ablation studies highlight the superiority of subgraph matching techniques for retrieving class-sharing probability, demonstrating significant improvements over baseline methods.

## 2. Related work

Recent surveys summarize the rapidly growing literature on heterophilic graph learning and provide broader perspectives on model design, benchmarks, and homophily metrics [33,34]. Graph neural networks (GNNs) for node classification are commonly grouped into spectral-based and spatial-based methods. Spectral-based methods exploit the structural information of the entire graph via Laplacian decomposition [35], but exact eigendecomposition is computationally expensive, with complexity $O(n^3)$. To reduce this cost, GCN [8] adopts a first-order approximation of Chebyshev polynomials [7], while Ada-GNN [36] learns adaptive frequency filters. However, these methods still aggregate neighborhood signals and may propagate noisy information when adjacent nodes are task-irrelevant or label-inconsistent.

Recent work also focuses on retrieving edge coefficients or redesigning propagation under heterophily. For example, GAT [10] measures the relevance between two nodes through attention, Masked-GCN [13] estimates attribute-wise similarity for precise

propagation, and GNNExplainer [16] identifies important edges and features that maximize the mutual information of the final prediction. To better cope with heterophily, FAGCN [14] allows negative coefficients to preserve high-frequency signals, L2Q [22] parameterizes the aggregation boundary of each node, and SuperGAT [15] differentiates between friendly and noisy neighbors. More recent heterophily-aware models further broaden this direction: GloGNN [37] learns global signed relations, Signed GNN [38] explicitly organizes multi-hop messages to alleviate heterophily and over-smoothing, LSGNN [39] uses local similarity to fuse multi-hop information adaptively, and LRGNN [40] predicts a global label-relationship matrix for propagation. Although effective, most of these methods estimate affinities within end-to-end GNN training and thus remain coupled with propagated node embeddings.

Some studies instead perform graph sparsification or denoising [41,42]. For instance, PTDNet [18] adopts nuclear norms to prune edges between communities. Yet sparsification also risks removing helpful assortative edges and may not always improve node classification. As another branch, non-local neural networks [43,44] and similarity-based GNNs aim to capture informative distant nodes beyond local adjacency. Geom-GCN [19] exploits structural neighborhoods within a bounded geometric space, while Simp-GCN [21] mixes the original adjacency matrix with a feature-based similarity matrix. SN-GNN [45] further leverages node-wise similarity matrices to guide edge-level propagation, and GREET [46] discriminates homophilic and heterophilic edges for unsupervised representation learning. These approaches improve access to non-local information, but measuring relevance mainly from feature similarity or propagated embeddings can still be brittle under scarce labels [47].

Recent benchmark studies also show that progress on heterophilic node classification should be interpreted carefully. In particular, duplicated nodes in Chameleon and Squirrel can induce train-test leakage and distort evaluation results [48]. Moreover, follow-up work argues that adjusted homophily and label informativeness provide a more faithful characterization than a single homophily ratio [49], while recent theory shows that the effect of heterophily depends jointly on neighborhood distributions, node degree, and propagation depth [50]. These observations motivate methods that explicitly inspect richer local structures.

In addition to retrieving edge coefficients, a strategy for using this information is important. For example, P-reg [51] simply uses entire edges to provide additional information for GNNs. NGM [32] integrates label propagation (LP) with GNNs, while GAM [27] further parameterizes edge coefficients. However, these methods are highly localized and do not explicitly compare broader neighborhoods when assessing edge reliability. In contrast, we focus on pairwise matching between two subgraphs that is independent of the backbone GNN. Using optimal transport (OT) [30,31,52,53], we integrate a confidence-based denoising module to estimate edge reliability, followed by sign-aware label propagation. This design emphasizes subgraph-level affinity estimation before message passing, which is especially beneficial when local neighbors are structurally informative but label-disassortative.

## 3. Notations

We first set up the problem with the commonly used notations of graph-structured data. Let us define an undirected graph $G = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V} = \{v_1, v_2, \dots, v_N\}$ represents a set of $N$ nodes and $\mathcal{E} = \{e_1, e_2, \dots, e_M\}$ denotes the set of $M$ edges. The adjacency matrix of graph $G$ is represented as $A \in \mathbb{R}^{N \times N}$, where $A_{ij} = 1$ if there is an edge between nodes $v_i$ and $v_j$, and $A_{ij} = 0$ otherwise. The properties of all nodes $N$ are encapsulated in the feature matrix $X \in \mathbb{R}^{N \times F}$, where $F$ is the number of features per node. Each row $X_i$ of the matrix $X$ corresponds to the feature vector of node $v_i$. We also define the label matrix $Y \in \mathbb{R}^{N \times C}$, where $C$ is the number of classes. Each row of $Y$ corresponds to a one-hot vector representing the label of node $v$. For instance, if node $v_i$ belongs to class $c$, then the $i$th row of $Y$ will have a 1 in the $c$th position and 0s elsewhere. Using a subset of partially labeled nodes $\mathcal{V}_L \subset \mathcal{V}$ as a training set, our objective is to infer the labels of the unlabeled nodes $\mathcal{V}_U = \mathcal{V} - \mathcal{V}_L$. For training, we use the labeled nodes in $\mathcal{V}_L$ to learn a mapping from the input features and graph structure to the node labels. This learned mapping is then used to predict the labels of nodes in $\mathcal{V}_U$. The training process involves optimizing a loss function that measures the discrepancy between predicted and true labels.

## 4. Methodologies

Fig. 2 illustrates the overall architecture of our model, which consists of two main components. On the right side, we describe the GNN module with label propagation, which utilizes the predicted edge weights for training. The left side shows the subgraph matching module, which provides edge coefficients as supplementary information. The two modules operate independently, with no shared loss functions or parameters, and are updated separately. In Section 4.1, we first introduce the methods for retrieving edge coefficients, then provide a detailed explanation of our subgraph matching module. In Section 4.2, we propose strategies to effectively use these edge-coefficient predictions via label propagation.

### 4.1. Retrieving edge coefficients

Recently, significant efforts have focused on specifying edge coefficients, and these methods can be categorized into two types. In Section 4.1.1, we review previous methods that use only central nodes for classification, focusing on their approaches and effectiveness. These methods typically rely on the intrinsic properties of the central nodes and do not account for broader graph context, which can limit their ability to capture the complexities of graph structures accurately.

In Section 4.1.2, we describe previous algorithms that extend beyond central nodes to incorporate adjacent nodes for prediction. These approaches leverage local neighborhood information to enhance predictive power by considering interactions between nodes. While this can improve accuracy, it also introduces additional complexity and potential noise, especially in heterophilic graphs where adjacent nodes may have different labels.

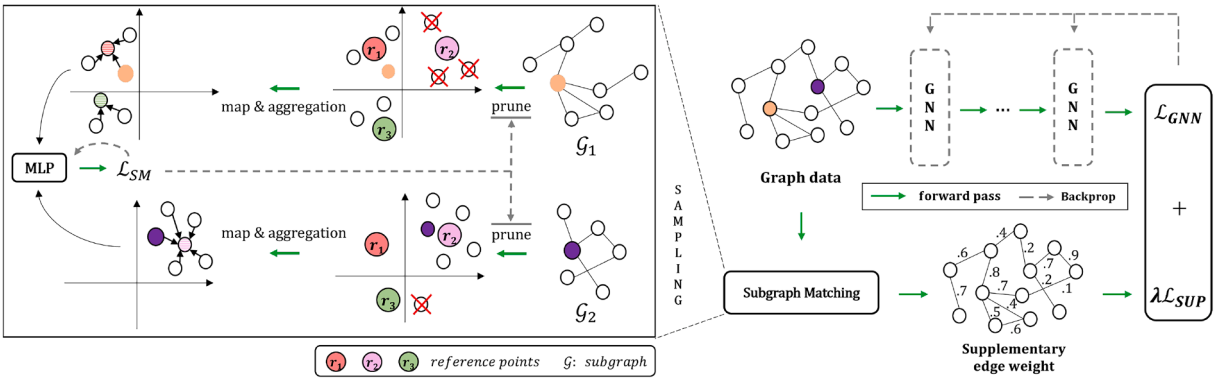


**Fig. 2.** The overall framework of our model. It consists of two parts: one for the subgraph matching module, which generates supplementary edge coefficients, and the other is the GNN module that utilizes weights for label propagation.

Finally, in Section 4.1.3, we discuss the advantages and limitations of these existing methods and present our subgraph matching module. Our approach aims to address the shortcomings of previous techniques by integrating a more comprehensive analysis of subgraph structures. By employing optimal transport and confidence-based mechanisms, our module provides robust estimates of edge coefficients, thereby enhancing the overall performance of the GNN.

#### *4.1.1. Retrieving edge coefficients using a central node*

These methods include message passing but use only the central node for similarity measures, not the entire subgraph. With a slight abuse of notation, let us denote $h_i$ and $h_j$ as the hidden representations of nodes $i$ and $j$.

**Graph Agreement Model (GAM)** [27] introduces an auxiliary model to predict the same class probability $w_{ij}$ between two nodes $i$ and $j$ as follows:

$$w_{ij} = MLP((h_i - h_j)^2). \tag{1}$$

Here, $MLP$ is a fully connected network with non-linear activation. GAM works well in heterophilic graph settings because it does not use neighboring nodes. However, as the graph's homophily ratio increases, GAM shows significantly lower performance, even compared with the plain GCN [8].

**Graph Attention Network (GAT)** [10] applies layer-wise attention as a downstream task of GNNs:

$$w_{ij}^l = \frac{exp(\sigma(a^l[h_i^l||h_j^l]))}{\sum_{k\in N_i} exp(\sigma(a^l[h_i^l||h_k^l]))}. \tag{2}$$

GAT specifies different weights for each layer, where $a^l$ is a learnable vector at the $l$th layer. Compared to GAM, a softmax function normalizes the weights, which are highly dependent on each node's degree, making it harder to determine their importance. Furthermore, message passing can degrade performance because the edge coefficients $w_{ij}$ are always positive.

**FAGCN** [14] improves GAT from two perspectives: replacing softmax with degree-based normalization and adopting a different activation function:

$$w_{ij}^l = tanh(a^l[h_i^l||h_j^l]). \tag{3}$$

The main difference lies in the use of $tanh$, which allows negative coefficients to preserve high-frequency signals. However, their accuracy decreases as network homophily increases (e.g., on the Cora dataset), where all coefficients converge to positive values, failing to identify heterophilic edges.

**PTDNet** [18] removes task-irrelevant edges by applying randomness $\epsilon$ and a decaying factor $\gamma$. The coefficients $w_{ij}$ are derived as follows:

$$w_{ij}^l = \sigma((\log \epsilon^l - \log(1-\epsilon^l) + MLP(h_i^l, h_j^l))/\gamma). \tag{4}$$

The random value follows $\epsilon^l \sim Uniform(0,1)$, and the decaying factor $\gamma$ depends on the iteration number. They apply a nuclear norm to the entire edge weights $w$ to remove connections between communities. However, randomness can hinder precise prediction, and the nuclear norm does not always yield optimal results.

Summarizing the above methodologies, prediction based on the central node implicates two major problems. Firstly, excluding message passing (as in GAM) can lead to over-fitting due to limited information. Although other methods incorporate neighboring nodes, noisy neighbors also participate in the aggregation process, which can undermine robustness and lead to over-smoothing [54]. Secondly, directly employing the coefficients as an adjacency matrix is highly risky, as the elimination of assortative edges can significantly hurt the overall performance of GNNs (refer to Fig. 1). To address these limitations, we focus on subgraph matching algorithms, which are introduced in the next section.

#### 4.1.2. Retrieving edge coefficients using subgraphs

In this section, we describe several methods for measuring the similarity between two subgraphs. Recently, applying Optimal Transport (OT) on subgraphs [28,29] has shown great improvement. OT is a mathematical framework for measuring distances (similarity) between objects. For example, let us assume two subgraphs $\mathcal{G}_i, \mathcal{G}_j$ that contain $m, n$ nodes, respectively. We can define a transport (coupling) matrix $P \in \mathcal{R}^{m \times n}$ between the two subgraphs that satisfies $P 1_n = \frac{1}{m} 1_m$ and $P^T 1_m = \frac{1}{n} 1_n$. The objective of OT is to find a matrix $P$ that minimizes the function below:

$$\min_P \sum_{ij} S(\mathcal{G}_i, \mathcal{G}_j) P_{ij} - \epsilon H(P). \tag{5}$$

Here, $S$ is a cost function, and $H(\cdot)$ is the entropy-regularized Kantorovich relaxation with regularizer $\epsilon$. However, finding $P_{ij}$ for all pairs of $(i, j)$ requires a high computational cost.

**Linear Optimal Transport** [30] employs reference points $r$ to address this limitation, which can be retrieved through $k$-means clustering or by calculating the Wasserstein barycenter [55] based on each class of training nodes. Here, elements assigned to the same cluster (reference) are pooled, reducing pairwise calculations. Specifically, matrix $P \in \mathcal{R}^{C \times N}$ splits or assigns the entire node set $N$ to references $r$ ($C$ stands for the number of reference points). $P$ can be obtained through various methods (e.g., Sinkhorn's algorithm [56]), which calculates the relevance between the inputs and reference points as below:

$$p_i^* = \arg\min_{p \in P_i} \sum_{k=1}^{C} \sum_{l=1}^{N_i} p_{kl} \|r_k - h_l\|^2, \tag{6}$$

where $N_i$ is the number of nodes in subgraph $i$.

Let us assume the hidden representations of two subgraphs as $h_i \in \mathcal{R}^{N_i \times F}$ and $h_j \in \mathcal{R}^{N_j \times F}$, where the feature dimension is $F$. Using $p_i^* \in \mathcal{R}^{C \times N_i}$ and $p_j^* \in \mathcal{R}^{C \times N_j}$ from Eq. (6), which splits the mass of subgraph $h_i$ and $h_j$ to multiple references $h_i'$ and $h_j' \in \mathcal{R}^{C \times F}$, one can measure their similarity through the matching function $M$ (implemented as an MLP) as below:

$$\begin{aligned} h_i' = p_i^* h_i, \;\; h_j' = p_j^* h_j \\ w_{ij} = M(h_i', h_j'). \end{aligned} \tag{7}$$

This method significantly reduces computational complexity by leveraging reference points, which serve as intermediate representations that capture the essential characteristics of subgraphs. Consequently, it enables efficient computation of subgraph similarities, making it feasible to apply OT at a large scale.

Moreover, optimal transport frameworks can be further enhanced with advanced techniques. For instance, incorporating hierarchical clustering can provide a multi-scale perspective that captures both local and global structural features. This approach allows the model to adaptively focus on different levels of granularity depending on the complexity of the graph structure.

In addition to computational efficiency, the use of reference points and clustering contributes to the model's robustness. By aggregating nodes into clusters, the model can mitigate the impact of noise and outliers, which is particularly beneficial in heterogeneous or noisy graph environments. This robustness is crucial for ensuring that the retrieved edge coefficients accurately reflect the underlying relationships between subgraphs. Thus, retrieving edge coefficients using subgraph-based methods, particularly those employing optimal transport, provides a powerful and scalable approach to capturing similarities between subgraphs. This technique not only enhances the computational efficiency but also improves the robustness and accuracy of the edge coefficient estimation, providing a solid foundation for subsequent label propagation and other graph-based learning tasks.

**Monge Map** [31] does not split the mass, but instead applies an injective mapping for each subgraph as follows:

$$\begin{aligned} h_i' = B(p_i^*, h_i), \;\; h_j' = B(p_j^*, h_j) \\ w_{ij} = M(h_i', h_j'). \end{aligned} \tag{8}$$

Similar to linear optimal transport [30], each subgraph $h_i, h_j$ can be mapped to new points $h_i', h_j' \in \mathcal{R}^{C \times F}$ through optimal transport $p^*$ (refer to Eq. (6)). The difference lies in a barycentric projection $B$ that ensures no mass splitting (refer to [31] for more details).

Using insights from these methods [30,31], subgraph matching benefits from utilizing adjacent nodes for predictions. However, these methods also face the limitation of handling noisy neighbors since they use all nodes within the subgraph to measure similarity. To address this issue, we introduce our method that incorporates a confidence ratio as described below.

#### 4.1.3. Our subgraph matching using a confidence ratio

In Fig. 2, we illustrate the overall architecture of our Confidence-based Subgraph Matching (ConSM). ConSM calculates the similarity between two subgraphs (probability of sharing the same label) using optimal transport and a confidence ratio as follows:

1. Sampling: We randomly sample two labeled nodes, which may have the same or different labels. To ensure balanced sampling, the number of positive and negative pairs should be equal to avoid model bias. We further use their 2-hop adjacent nodes as inputs to capture a more comprehensive local structure.

2. Prune: We assess the score of all edges through reference points. Based on a confidence ratio, we retain the top-$k$ confident edges while discarding the others. This step is crucial for reducing noise and focusing on the most relevant connections.

3. Map and Aggregation: Given two subgraphs $\mathcal{G}_i, \mathcal{G}_j$, we first map nodes to low-dimensional embeddings $h_i, h_j$ and assign them to the nearest reference points $r$ using the Monge map. We then aggregate the nodes that belong to the same reference points using a pooling operation (e.g., mean), which simplifies the representation while preserving essential information.

4. Prediction: We measure the similarity between the two subgraphs and also retrieve the class probability of a central node. This step involves using a matching function $M$ to compute the similarity score $w_{ij}$, which reflects the likelihood that the two subgraphs share the same label.

#### 4.1.4. Details of each step

**1. Sampling** We adopt an auxiliary module for retrieving edge coefficients that are independent of the GNN module. Here, two nodes are randomly sampled from their respective classes. If two nodes share the same class (positive pair), we label the pair 1; otherwise, we label it 0. To prevent class imbalance problems, we ensure that the same number of positive and negative pairs are sampled. Unlike GAM [27], we utilize the subgraph of a central node (adjacent nodes within 2-hop) to improve prediction accuracy by providing more contextual information.

**2. Prune** Unlike previous methods [31] that applied *embedding* → *aggregation* → *mapping*, we suggest *embedding* → *pruning* → *mapping* → *aggregation* to handle noisy edges. Specifically, we use only a subset of edges based on their scores and a confidence ratio ($\zeta$). We first describe our scoring function. Using the initial node features $X$, we retrieve their low-dimensional embedding $h \in \mathcal{R}^{N\times F}$ through an encoder (MLP):

$$h = Encoder(X). \tag{9}$$

Similarly, the embedding of reference points is given by $r \in \mathcal{R}^{C\times F}$, obtained by class-wise averaging across training nodes. We then measure a score (e.g., cosine similarity) $S \in \mathcal{R}^{N\times C}$ between nodes $h$ and references $r$ as follows:

$$S = h \cdot r^T, \tag{10}$$

where each row of $S$ represents the score of each node concerning the reference points. Given two nodes $i$ and $j$, we retrieve their similarity $w_{ij} = S_i \cdot S_j$. Consequently, $w$ for all edges is obtained, and we maintain the top-$k$ edges ($k = \lfloor \zeta \times |\mathcal{E}| \rfloor$) while removing others.

**3. Map and Aggregation** Using the remaining edges, the adjacency matrix is reconstructed. With a slight abuse of notation, given two nodes $i$ and $j$ and their subgraphs $\mathcal{G}_i$ and $\mathcal{G}_j$, we assume their subgraph embeddings $h_i$ and $h_j$ are retrieved as below:

$$h_i = Encoder(\mathcal{G}_i),\ \ h_j = Encoder(\mathcal{G}_j), \tag{11}$$

similar to Eq. (9). Here, $h_i \in \mathcal{R}^{m\times F}$ and $h_j \in \mathcal{R}^{n\times F}$ consist of $m$ and $n$ nodes, respectively. Referring to Eqs. (6) and (8), we map each node in the subgraph through the Monge map as follows:

$$h'_i = B(p_i^*, h_i),\ \ h'_j = B(p_j^*, h_j). \tag{12}$$

The $h'_i$ and $h'_j \in \mathcal{R}^{C\times F}$ are the outputs of the subgraph after mapping and aggregation. Though linear OT (Eq. (7)) is also considerable, we choose the Monge Map due to its superior performance.

**4. Prediction** Finally, using the concatenation of $h'_i$ and $h'_j$ as an input to the matching function $M$, we estimate their similarity:

$$w_{ij} = M(h'_i \oplus h'_j). \tag{13}$$

If the two inputs share the same label, the value of $w_{ij}$ should be closer to 1; otherwise, it should be closer to 0. Additionally, we employ a node classification function $f(\cdot)$ (MLP) to predict the label of each subgraph's central node $h_i^e$ and $h_j^e$, where $\mathcal{L}_{nll}$ is the negative log-likelihood function:

$$\mathcal{L}_{SM} = \sum_{Y_i \neq Y_j} |w_{ij}| + \sum_{Y_i = Y_j} |w_{ij} - 1| + \mathcal{L}_{nll}(f(h_i^e), Y_i) + \mathcal{L}_{nll}(f(h_j^e), Y_j). \tag{14}$$

This comprehensive framework ensures that only the most relevant and confident edges are used for prediction, enhancing the robustness and accuracy of our model. By leveraging the insights from optimal transport and incorporating a confidence-based pruning mechanism, ConSM effectively mitigates the impact of noisy neighbors and improves the reliability of subgraph matching. Our subgraph matching module can be trained through Eq. (14), and we describe the overall procedure in Algorithm 1.

### 4.2. GNNs with supplementary edge weights

Recent studies focus on strategies to better utilize edge weights. For example, some methods directly construct the adjacency matrix [10,14,18], while others employ label propagation (LP) [27,32,57] on GNNs to handle uncertainty as follows:

$$\mathcal{L} = \mathcal{L}_{GNN} + \lambda \mathcal{L}_{LP}. \tag{15}$$

$\mathcal{L}_{GNN}$ is a widely used loss function for semi-supervised node classification (e.g., GCN [8]), defined as follows:

$$\begin{aligned} \widehat{Y} &= softmax(\widehat{A}\sigma(\widehat{A}XW_0)W_1), \\ \mathcal{L}_{GNN} &= \mathcal{L}_{nll}(Y, \widehat{Y}), \end{aligned} \tag{16}$$

where $W$ is a learnable matrix. Though many recently proposed methods [10,14,58–60] are considerable for GNNs, we select GCN to demonstrate the efficacy of our method. Returning to Eq. (15), $\lambda$ is a regularizer, and $\mathcal{L}_{LP}$ provides additional penalties as follows:

$$\mathcal{L}_{LP} = \alpha_1 \sum_{i,j\in LL} w_{ij} d(i,j) + \alpha_2 \sum_{i,j\in LU} w_{ij} d(i,j) + \alpha_3 \sum_{i,j\in UU} w_{ij} d(i,j). \tag{17}$$

**Algorithm 1** Confidence-based subgraph matching.

**Require:** Adjacency matrix $A$, initialized parameters $\theta_{SM}$, number of entire training epochs $K_e$, learning rate $\eta$
**Ensure:** Supplementary edge coefficients $w$
1: **for** number of entire training epochs $K_e$ **do**
2: Get embedding of nodes through Eq. (9)
3: Get reference points $r$ by averaging embedding of training nodes per each class
4: Find top-k confident edges through Eq. (10)
5: Using the confident edges, reconstruct adjacency matrix within 2-hop as $\tilde{A} = A \vee A^2$
6: Sampling positive or negative node pair $(i, j)$
7: **for** each node pair $(i, j)$ **do**
8: Find the subgraphs $\mathcal{G}_i = \tilde{A}[i]$, $\mathcal{G}_j = \tilde{A}[j]$, and obtain their embedding $h_i, h_j$ through Eq. (11)
9: Apply Monge map for each subgraph Eq. (12)
10: Retrieve the edge coefficient through Eq. (13)
11: Compute the loss $\mathcal{L}_{SM}$ using Eq. (14)
12: Update $\theta'_{SM} = \theta_{SM} - \eta \frac{\partial \mathcal{L}_{SM}}{\partial \theta_{SM}}$
13: **end for**
14: **end for**
15: Calculate supplementary edge coefficients $w$ using $\theta'_{SM}$

The notation $\{L, U\}$ denotes labeled and unlabeled nodes, whereas $LU$ means that only one node is labeled. $w_{ij} \in \{0, 1\}$ is a binary value representing a connection between two nodes $i$ and $j$, and $d$ is a dissimilarity measuring function (e.g., cosine similarity). Here, $\{\alpha_1, \alpha_2, \alpha_3\}$ act as hyper-parameters.

Recently, the graph agreement model (GAM) [27] addresses the limitation of a fixed $w_{ij}$ by replacing it with a parameterized model $w_{ij} = g(X_i, X_j)$, where $g$ is a fully connected network. However, these methods are limited by two limitations. Firstly, they have shown inferior performance in discriminating task-irrelevant edges under semi-supervised learning. Secondly, the estimated $w_{ij}$ ranges from zero to one, which makes disassortative nodes appear similar (P-reg [51] also has this limitation). Thus, we improve Eq. (17) as follows:

$$\mathcal{L}_{SUP} = \alpha_1 \left( \sum_{i,j \in LU, w_{ij} > k} w_{ij} d(i,j) + \sum_{i,j \in LU, w_{ij} \leq k} (1 - w_{ij})(1 - d(i,j)) \right) + \alpha_2 \left( \sum_{i,j \in UU, w_{ij} > k} w_{ij} d(i,j) + \sum_{i,j \in UU, w_{ij} \leq k} (1 - w_{ij})(1 - d(i,j)) \right). \tag{18}$$

By sorting the scores of all edges $w$, we retrieve a threshold $k$ based on a given confidence ratio. In Eq. (18), weights $w_{ij}$ greater than $k$ are trained to reduce dissimilarity $d(i, j)$, while the others are trained to maximize dissimilarity $1 - d(i, j)$. Referring to Eq. (15), we replace $\mathcal{L}_{LP}$ with our $\mathcal{L}_{SUP}$ and define $\mathcal{L}_G$ as follows:

$$\mathcal{L}_G = \mathcal{L}_{GNN} + \lambda \mathcal{L}_{SUP}. \tag{19}$$

As described in GAM [27], we exclude edges between labeled nodes ($i, j \in LL$) and set $\alpha_1 = 1.0$, $\alpha_2 = 0.5$. We set $0.01 \leq \lambda \leq 0.1$, which is proportional to the dataset's disassortativity.

### 4.3. Optimization strategy

So far, we have defined the losses of our subgraph matching with label propagation in Eqs. (14) and (19). Let us assume the parameters of the subgraph matching module are $\theta_{SM}$ and those of the GNN module are $\theta_G$, without sharing parameters. Here, we note that our ConSM imposes two limitations on optimization. Firstly, it is challenging to determine whether the subgraph matching module $\theta_{SM}$ has converged. Secondly, the predicted edge coefficients may imply uncertainty, impeding the training of GNNs.

To address this, in Algorithm 2, we suggest saving parameters $\theta'_G$ only if they attain the best validation score (line 13). Then, before computing the loss of the next training sample, we can load these parameters if they exist (line 14). Through this mechanism, we guide GNNs to achieve better performance despite the uncertainty of supplementary weights.

**Computational Complexity Analysis**

The computational costs of our model can be divided into two parts. The first part is the vanilla GCN [8] model, whose complexity is known as $\mathcal{O}(|\mathcal{E}|P_{GCN})$, where the costs are proportional to the number of edges $|\mathcal{E}|$ and the size of the learnable matrices $P_{GCN}$. The second part is our ConSM, which computes the similarity between two subgraphs. Instead of naively calculating the Wasserstein distance, which has a complexity of $\mathcal{O}(n^3 \log(n))$, we employ linear mapping [31] and measure the Euclidean distance, which can be retrieved through simple matrix multiplication. Consequently, our computational cost can be defined as $\mathcal{O}(|\mathcal{E}|P_{GCN} + |\mathcal{E}_M|P_{ConSM})$,

**Algorithm 2** Overall optimization of ConSM.

**Require:** Initialized parameters $\theta_{SM}$ and $\theta_G$, number of entire training epochs $K_e$, best validation score $\beta' = 0$, learning rate $\eta$
**Ensure:** Trained parameters $\theta'_{SM}$, $\theta'_G$

**for** number of entire training epochs $K_e$ **do**
  **for** training samples **do**
    Compute the $\mathcal{L}_{SM}$ using Eq. (14)
    Update $\theta'_{SM} = \theta_{SM} - \eta \frac{\partial \mathcal{L}_{SM}}{\partial \theta_{SM}}$
  **end for**
  Retrieve edge coefficients $w$ using $\theta'_{SM}$
  **for** training samples **do**
    Compute the $\mathcal{L}_G$ using Eq. (19)
    Compute the validation score $\beta$
    Update $\theta'_G = \theta_G - \eta \frac{\partial \mathcal{L}_G}{\partial \theta_G}$
    **if** $\beta > \beta'$ **then**
      Save updated parameters $\theta'_G$
      $\beta' = \beta$
    **end if**
  **end for**
  Load $\theta'_G$ if exists
**end for**

where $|\mathcal{E}_M|$ stands for the number of edges included in the sampled subgraph, and $P_{ConSM}$ represents the set of parameters in the subgraph matching module.

Our approach is designed to be efficient, leveraging the linear mapping to reduce the computational overhead significantly. By utilizing optimal transport with a confidence ratio, we minimize unnecessary calculations, focusing only on the most relevant edges. This balance between accuracy and efficiency ensures that ConSM remains scalable and practical for large-scale graph datasets.

## 5. Theoretical analysis

We provide a principled analysis for three key design choices of ConSM: (i) why edge coefficients learned via subgraph-level structural similarity alleviate heterophily better than feature-only measures; (ii) why using 2-hop egonets strikes an advantageous trade-off between expressivity and noise; and (iii) how the confidence ratio $\zeta$ influences performance and how our adaptive strategy mitigates its sensitivity.

### 5.1. Edge coefficients as probabilities of label agreement

Let $G = (\mathcal{V}, \mathcal{E})$ be a graph with node labels $Y = \{y_i\}_{i \in \mathcal{V}}$. For an edge $(i, j) \in \mathcal{E}$, define

$$\theta_{ij} := \mathbb{P}(y_i = y_j \mid \mathcal{I}), \tag{20}$$

where $\mathcal{I}$ denotes the information used to infer edge affinity (e.g., node features, subgraph structure). In homophilic graphs, $\mathbb{P}(y_i = y_j \mid (i,j) \in \mathcal{E})$ is naturally high; under heterophily it can be arbitrarily low. Our edge coefficient $w_{ij}$ aims to estimate $\theta_{ij}$ as closely as possible:

$$w_{ij} \approx \mathbb{P}(y_i = y_j \mid \Phi(\mathcal{G}_i^{(k)}), \Phi(\mathcal{G}_j^{(k)})), \tag{21}$$

where $\mathcal{G}_i^{(k)}$ is the $k$-hop egonet of $i$ and $\Phi(\cdot)$ is a structural embedding.

*Feature-only vs structure-aware.* Feature-only approaches use $\mathcal{I} = \{X_i, X_j\}$, which becomes unreliable when connected nodes with dissimilar labels also possess dissimilar features (common in heterophily). By instead conditioning on structural descriptors $\Phi(\mathcal{G}_i^{(k)})$, we capture what we call pattern-level homophily: even if $X_i$ and $X_j$ differ, their local distributions of node types or structural motifs may still be similar, increasing $\theta_{ij}$.

**Proposition 1** (Risk of Misleading Aggregation). *Consider a message-passing GNN that aggregates neighbor features with weights $\alpha_{ij}$. Let $\ell(\cdot)$ be a convex loss on predictions. The expected loss on node $i$ can be decomposed as follows:*

$$\mathbb{E}[\ell(\hat{y}_i, y_i)] \le C_1 \sum_{j \in \mathcal{N}(i)} \alpha_{ij}(1 - \theta_{ij}) + C_2, \tag{22}$$

*for constants $C_1, C_2 > 0$ (dependent on $\ell$ and model capacity). Thus, over-weighting edges with small $\theta_{ij}$ increases risk.*

The proposition formalizes the intuition: if $\alpha_{ij}$ tracks $\theta_{ij}$ (high when $\theta_{ij}$ is high, low or negative otherwise), aggregation risk is reduced. Our $w_{ij}$, derived from subgraph similarity, is a surrogate for $\theta_{ij}$. Empirically, we show its F1 against true edge labels is higher than feature-only baselines.

### 5.2. Why subgraphs, and why 2-hop?

Let $p_r := \mathbb{P}(y_u = y_v \mid u \text{ and } v \text{ are } r\text{-hop apart via some path})$. In many real graphs, $p_1$ (direct neighbors) can be lower than $p_2$ due to heterophily, but $p_r$ may again deteriorate for large $r$ as paths cross multiple communities. We capture this with a simple stylized model.

For each node $i$, there exists a radius $r^\star \in \{1, 2, 3, \dots\}$ such that the label distribution of its $r^\star$-hop neighbors is more predictive of $y_i$ than that of its $(r^\star - 1)$- or $(r^\star + 1)$-hop neighbors, i.e.,

$$\mathrm{MI}\left(y_i; \mathcal{G}_i^{(r^\star)}\right) \geq \max\left\{\mathrm{MI}\left(y_i; \mathcal{G}_i^{(r^\star-1)}\right), \mathrm{MI}\left(y_i; \mathcal{G}_i^{(r^\star+1)}\right)\right\}, \tag{23}$$

where MI denotes mutual information. Empirically, we observe $r^\star = 2$ for most benchmarks: 1-hop is too myopic (high heterophily, low MI), while $r \geq 3$ brings in many noisy/unrelated nodes, diluting MI. Thus, 2-hop egonets offer the best accuracy-to-cost ratio.

**Proposition 2** (Complexity vs Noise Trade-off). *Let $|\mathcal{G}_i^{(r)}|$ denote the size of an r-hop egonet. The cost of OT-based matching between two egonets scales as $\tilde{\mathcal{O}}(|\mathcal{G}_i^{(r)}| + |\mathcal{G}_j^{(r)}|)$ after linearization. If the expected noise proportion $\eta(r)$ grows superlinearly with r while $\mathrm{MI}(y_i; \mathcal{G}_i^{(r)})$ saturates at $r = 2$, then the expected generalization benefit $\Delta(r)$ is maximized at $r = 2$.*

The above proposition states that beyond 2 hops, the marginal gain in useful signal is outweighed by computational and noise costs.

### 5.3. Confidence ratio $\zeta$: sensitivity and adaptive mitigation

We define $\zeta \in (0, 1]$ as the quantile used to split edges into attractive ($w_{ij} > k_\zeta$) and repulsive ($w_{ij} \leq k_\zeta$) sets, where $k_\zeta$ is the $\zeta$th order statistic of $\{w_{ij}\}$. Let $\mathcal{L}_{\mathrm{SUP}}(\zeta)$ denote our sign-aware label propagation loss and $\mathcal{L}_{\mathrm{val}}(\zeta)$ be the validation loss.

**Lemma 1** (Piecewise Smoothness). *If the distribution of $\{w_{ij}\}$ is continuous, then $\mathcal{L}_{SUP}(\zeta)$ is piecewise smooth in $\zeta$, with kinks only when $k_\zeta$ crosses a distinct $w_{ij}$. Thus, small perturbations in $\zeta$ do not drastically change $\mathcal{L}_{SUP}$ unless they swap many edges across the threshold.*

Despite Lemma 1, in practice $\{w_{ij}\}$ can be multi-modal, making $\zeta$ seemingly sensitive. We therefore adopt an adaptive scheduler that aligns the fraction of attractive edges with an empirical homophily estimate $h_{\mathrm{val}}$ on the validation set:

$$\zeta_{t+1} = \zeta_t + \eta\left(h_{\mathrm{val}} - \hat{h}_t\right), \quad \hat{h}_t := \frac{1}{|\mathcal{E}_{\mathrm{val}}|} \sum_{(i,j)\in\mathcal{E}_{\mathrm{val}}} \mathbf{1}\{w_{ij} > 0.5\}. \tag{24}$$

Here, $\eta > 0$ is a small step size. Intuitively, if our current set of high-confidence edges is smaller than the observed homophily, we increase $\zeta$.

**Theorem 1** (Convergence of Adaptive $\zeta$). *Assume $\mathcal{L}_{val}(\zeta)$ is Lipschitz around the optimum $\zeta^\star$ and $|\partial\hat{h}_t/\partial\zeta| \leq L_h$. Choosing $\eta \in (0, 2/L_h)$ yields a stable fixed point $\zeta_t \to \zeta^\star$ where $\hat{h}_t \approx h_{val}$, minimizing validation loss up to first order.*

This shows our adaptive rule is a simple bi-level optimizer: the inner loop updates model parameters; the outer loop updates $\zeta$ to minimize validation loss.

## 6. Experiments

In this section, we compare our ConSM with several state-of-the-art methods using both homophilic and heterophilic graph datasets. We aim to answer the following research questions:

- **RQ1:** Does ConSM improve node classification accuracy compared to state-of-the-art approaches?
- **RQ2:** How accurately does ConSM identify task-irrelevant edges for graph denoising?
- **RQ3:** How does the confidence ratio for the subgraph matching module affect the overall classification result?
- **RQ4:** Can ConSM effectively alleviate over-smoothing when stacking many layers?
- **RQ5:** Does the method also improve link-prediction performance?
- **RQ6:** If the number of training nodes increases, how does the performance of the proposed method change?
- **RQ7:** How does ConSM perform on leakage-free splits like Chameleon and Squirrel datasets?

### 6.1. Dataset description and baselines

**Dataset Description**: We conduct investigations with the following publicly available datasets. The statistical details are presented in Tables 1 and 2, where we categorize the networks into two types: assortative (homophilic) and disassortative (heterophilic). The explanations of each dataset are as follows:

Assortative Networks: For assortative data, we adopt widely used benchmark graphs: Cora, Citeseer, and Pubmed [8]. Here, each node represents a paper, and the edge denotes a citation between two papers. Node features represent the bag-of-words of the paper, and each node has a unique label based on its relevant topic. - Cora: A citation network of computer science papers. The nodes are papers, and the edges represent citations. There are seven classes, each representing a specific topic. - Citeseer: Another citation

**Table 1**
Statistical details of homophilic datasets.

| Datasets | Cora | Citeseer | Pubmed |
|---|---|---|---|
| # Nodes | 2708 | 3327 | 19,717 |
| # Edges | 10,558 | 9104 | 88,648 |
| # Features | 1433 | 3703 | 167,597 |
| # Classes | 7 | 6 | 3 |
| # Training Nodes | 140 | 120 | 60 |
| # Validation Nodes | 1568 | 2207 | 18,657 |
| # Test Nodes | 1000 | 1000 | 1000 |

**Table 2**
Statistical details of heterophilic datasets.

| Datasets | Actor | Chameleon | Squirrel |
|---|---|---|---|
| # Nodes | 7600 | 2277 | 5201 |
| # Edges | 25,944 | 33,824 | 211,872 |
| # Features | 931 | 2325 | 2089 |
| # Classes | 5 | 5 | 5 |
| # Training Nodes | 100 | 100 | 100 |
| # Validation Nodes | 3750 | 1088 | 2550 |
| # Test Nodes | 3750 | 1089 | 2551 |

network where nodes are research papers and edges represent citations. It contains six classes. - PubMed: A citation network from the biomedical literature. Each node is a paper, and each edge is a citation link. It includes three classes of diabetes research.

Disassortative Networks: We adopt the Actor co-occurrence graph [61] and Wikipedia network [24] as disassortative graphs. - Actor: The node stands for an actor, and the edges represent co-occurrence on the same Wikipedia pages. The node labels denote five types based on the keywords associated with the actor's profession. - Wikipedia Network: This includes Chameleon and Squirrel datasets, where the edges are hyperlinks between web pages. The node features are several informative nouns, and nodes are classified into five categories based on their monthly traffic. - Chameleon: A web page network where nodes represent web pages, and edges are hyperlinks. The labels are categorized based on the monthly traffic of the web pages. - Squirrel: Similar to Chameleon, but includes different web pages and hyperlink structures, also categorized based on monthly traffic. - Regarding Chameleon and Squirrel, a recent study [48] points out duplicated nodes, which may lead to information leakage.

Baselines: Using the above datasets, we compare our method with several state-of-the-art baselines. A brief explanation of these methods is as follows:

- **MLP** [62] employs a feedforward neural network that uses only a central node for classification.
- **GCN** [8] is a traditional GNN models that suggests first order approximation of Chebyshev polynomials [7] to localize spectral filters.
- **DropEdge** [63] randomly removes edges under a given probability to alleviate over-fitting problem.
- **GAT** [10] specifies different weights between two nodes, while ignoring graph Laplacian matrix.
- **GIN** [58] pointed out the limited discriminative power of GCN, suggesting a graph isomorphism network that satisfies the injectiveness condition.
- **APPNP** [59] combines personalized PageRank with GCN that improves prediction accuracy, while reducing computational complexity.
- **GCNII** [60] integrates identity mapping to redeem the deficiency of APPNP.
- **GAM** [27] adopts the graph agreement model under the assumption that not all edges correspond to sharing the same label between nodes.
- **$H_2$GCN** [64] suggests ego-neighbor separation and hop-based aggregation to deal with heterophilic graph.
- **FAGCN** [14] further utilizes high-frequency signal beyond low-frequency information in GNNs.
- **PTDNet** [18] proposes a topological denoising network to prune task-irrelevant edges as a downstream task of GNNs.
- **GloGNN** [37] introduces a global homophily signal that helps select more reliable neighbor information across distant nodes.
- **SN-GNN** [45] leverages node-wise similarity matrices to guide edge-level propagation paths, particularly effective for heterophilic graphs.
- **GREET** [46] performs unsupervised graph representation learning by explicitly identifying edge-level heterophily and adjusting smoothing accordingly.

### 6.2. Experimental setup

All methods are implemented in *PyTorch Geometric*,[1] using the Adam optimizer with a weight decay of $5e^{-4}$ and a learning rate of $1e^{-3}$. We set the embedding dimension to 64 for all methods, though diversifying it could further improve performance [65]. We adopt

[1] https://pytorch-geometric.readthedocs.io/en/latest/modules/nn.html

**Table 3**
(RQ1) Node classification accuracy (%) on homophilic citation networks. Bold* symbol indicates the best performance, and methods with † are built upon GCN.

| Datasets<br>Hom. ratio ($h$) | Cora<br>0.81 | Citeseer<br>0.74 | Pubmed<br>0.8 |
|---|---|---|---|
| MLP | 54.2 ±0.5% | 53.7 ±1.7% | 69.7 ±0.4% |
| GCN | 80.5 ±0.4% | 67.5 ±0.6% | 78.4 ±0.2% |
| DropEdge† | 80.6 ±0.4% | 68.4 ±0.5% | 78.3 ±0.3% |
| GAT | 81.4 ±0.4% | 69.3 ±0.9% | 78.6 ±0.5% |
| GIN | 78.2 ±0.2% | 65.1 ±0.6% | 76.8 ±0.8% |
| APPNP | 82.1 ±0.4% | 69.2 ±0.7% | 78.7 ±0.4% |
| GCNII | 81.1 ±0.3% | 67.0 ±0.4% | 78.5 ±0.8% |
| GAM† | 81.3 ±0.6% | 70.4 ±0.3% | 79.2 ±0.1% |
| $H_2$GCN | 80.5 ±0.2% | 68.9 ±0.5% | 78.9 ±0.2% |
| FAGCN | 81.5 ±0.5% | 68.6 ±0.2% | 79.0 ±0.1% |
| PTDNet† | 81.5 ±0.7% | 69.4 ±0.6% | 76.9 ±1.1% |
| GloGNN | 82.5 ±0.3% | 70.2 ±0.4% | 78.9 ±0.2% |
| SN-GNN | 81.7 ±0.4% | 69.8 ±0.3% | 78.7 ±0.3% |
| GREET | 80.8 ±0.2% | 68.7 ±0.4% | 78.1 ±0.3% |
| Ours† | **83.6*** ±0.3% | **71.2*** ±0.2% | **79.6*** ±0.1% |

**Table 4**
(RQ1) Node classification accuracy (%) on heterophilic Wikipedia pages. Bold* symbol indicates the best performance, and methods with † are built upon GCN.

| Datasets<br>Hom. ratio ($h$) | Actor<br>0.22 | Chameleon<br>0.23 | Squirrel<br>0.22 |
|---|---|---|---|
| MLP | **27.9*** ±1.1% | 41.2 ±1.8% | 26.5 ±0.6% |
| GCN | 21.4 ±0.6% | 49.4 ±0.7% | 33.1 ±1.2% |
| DropEdge† | 21.9 ±0.4% | 49.2 ±0.8% | 31.0 ±1.4% |
| GAT | 23.2 ±0.8% | 48.2 ±0.6% | 30.1 ±0.9% |
| GIN | 23.6 ±0.5% | 40.1 ±3.7% | 23.0 ±2.4% |
| APPNP | 21.7 ±0.2% | 45.2 ±0.7% | 30.6 ±0.7% |
| GCNII | 25.7 ±0.4% | 45.1 ±0.5% | 28.8 ±0.3% |
| GAM† | 21.7 ±0.3% | 49.5 ±0.5% | 33.9 ±0.4% |
| $H_2$GCN | 22.8 ±0.3% | 46.6 ±1.2% | 29.8 ±0.7% |
| FAGCN | 26.9 ±0.4% | 46.7 ±0.6% | 29.5 ±0.7% |
| PTDNet† | 21.5 ±0.5% | 49.7 ±0.9% | 32.6 ±0.7% |
| GloGNN | 26.4 ±0.5% | 50.2 ±0.6% | 34.5 ±0.4% |
| SN-GNN | 25.8 ±0.7% | 50.7 ±0.5% | 34.0 ±0.6% |
| GREET | 24.9 ±0.4% | 48.8 ±0.8% | 33.2 ±0.5% |
| Ours† | 27.7 ±0.4% | **52.3*** ±0.4% | **35.7*** ±0.5% |

2 layers of GNNs for all baselines, while APPNP, GCNII, and GIN also utilize 2 layers of fully connected networks for classification. ReLU is applied as the activation function, except for PTDNet which uses Sigmoid. Softmax is applied to the last hidden layers for classification. For all datasets, we randomly select 20 samples per class as the training set, with the rest used for validation and testing (refer to Tables 1 and 2). Performance is evaluated based on test set accuracy that achieved the best validation score.

### 6.3. Results and discussion (RQ1)

In Tables 3 and 4, we present the experimental results of baselines and our method, conducted on homophilic and heterophilic datasets. We assume the homophily ratio $h$ as below:

$$h = \frac{\#\text{of edges that connect nodes with the same label}}{\#\text{of entire edges}} \tag{25}$$

**Results on Homophilic Graph Datasets:** In Table 3, we evaluate model performance on three homophilic citation networks (Cora, Citeseer, and Pubmed), where most connected nodes share the same label. Each experiment is repeated 10 times, and we report the mean and standard deviation of test accuracy.

We first observe that GCN-based variants (marked with †) generally perform well in these settings due to strong label consistency in the neighborhood. Our method achieves state-of-the-art performance across all three datasets, outperforming vanilla GCN by 3.1 %, 3.7 %, and 1.2 % on Cora, Citeseer, and Pubmed, respectively.

Among baselines, APPNP, GAM, and GloGNN show strong performance thanks to their use of personalized propagation or global homophily alignment. In particular, GloGNN achieves competitive results on all three datasets, demonstrating its effectiveness even in homophilic regimes. SN-GNN and GREET also perform reasonably well, though they slightly lag behind the top models, possibly due to their design targeting more heterophilic graphs.

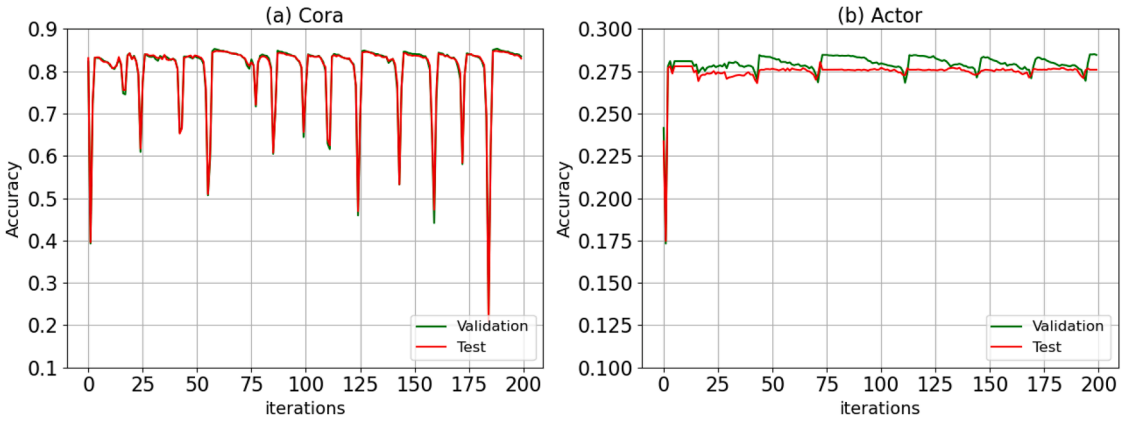


**Fig. 3.** (RQ1) Convergence analysis on (a) Cora, and (b) Actor. Each figure contains validation (green) and test (red) accuracy of node classification. (For interpretation of the references to colour in this figure legend, the reader is referred to the web version of this article.)

Interestingly, DropEdge improves over GCN in Citeseer, likely because its moderate homophily (0.74) benefits from structural regularization. GCNII underperforms APPNP, suggesting that emphasizing identity mapping may not be optimal when label propagation is already strong. Additionally, models tailored for heterophilic graphs ($H_2$GCN, FAGCN, PTDNet) fail to show significant gains over GCN in these datasets, highlighting the importance of matching model design to the graph's homophily level.

**Results on Heterophilic Graph Datasets**: We also evaluate all methods on heterophilic graphs under the same experimental settings, with results summarized in Table 4. These datasets exhibit low homophily ratios ($h \approx 0.22$), which generally undermines the effectiveness of traditional message-passing GNNs.

Interestingly, MLP achieves the best performance on *Actor*, implying that neighborhood information is noisy and less helpful in this case. Our method ranks a close second, followed by FAGCN and GCNII. The relatively strong performance of GCNII compared to APPNP suggests that retaining central-node identity plays a more significant role than iterative label propagation on this dataset.

On the other hand, our model achieves the best accuracy on both *Chameleon* and *Squirrel*, which are known for their high structural heterophily and feature inconsistency. ConSM outperforms strong baselines such as SN-GNN, GloGNN, and GREET, indicating that our subgraph-based edge filtering and coefficient learning mechanism better preserves informative connections in such noisy graphs.

Among prior heterophily-aware models, GAM performs consistently well across all three datasets, while SN-GNN and GloGNN show solid but slightly lower performance. GREET lags behind, possibly due to its reliance on fixed global propagation rules. Notably, $H_2$GCN, FAGCN, and PTDNet underperform across the board, which aligns with their design assumptions favoring shallow or frequency-filtered aggregation - less effective under strong disassortativity.

We further analyze these trends in Section 6.4, where we discuss robustness, ablations, and coefficient interpretability in depth.

**Convergence Analysis**: To better understand the convergence of ConSM, Fig. 3 depicts validation and test accuracy during training. The x-axis illustrates iterations, while the y-axis shows classification accuracy. As described in Algorithm 2, a single iteration combines training subgraph matching modules followed by training GNN layers. Here, the validation and test accuracy vary significantly, but ConSM manages to achieve better performance as iteration increases. This is because ConSM loads parameters with the best validation score (refer to Algorithm 2), which can precisely prevent the uncertainty of supplementary information.

### 6.4. Edge classification (RQ2)

To validate whether ConSM can predict edge coefficients correctly, we examine the accuracy of our method and several state-of-the-art approaches. We assume the label of edges connecting two nodes with the same class as 1, and vice versa. For each method, we sort their predicted coefficients and select the $k$th largest value as a threshold, equal to the number of positive edges. Specifically, for (a) Cora, $h = 0.81$ and $\mathcal{E} = 10,558$ (refer to Tables 1 and 3), thus $k = \lfloor 10,558 \times 0.81 \rfloor$, described in Fig. 4. We adopt the F1-score, which is effective for binary classification:

$$\text{F1-score} = \frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}} \tag{26}$$

We first introduce some details of the baselines, followed by a discussion of the experimental results. (1) GAM: As described in Eq. (1), the agreement model generates the same class probability. We use their edge coefficients with the best validation result. (2) GAT: We exclude node-wise normalization (e.g., softmax), which can be highly sensitive to the degree of central nodes. Using the final hidden layer representations, we retrieve the attention value of all edges. Multi-head attention is applied for the initial layers, while the final layer employs single-head attention. (3) FAGCN: They retrieve coefficients following Eq. (3). Similar to GAT, we use the attention values of the last hidden representations. Hyper-parameters are tuned according to [14]. (4) PTDNet: We use the

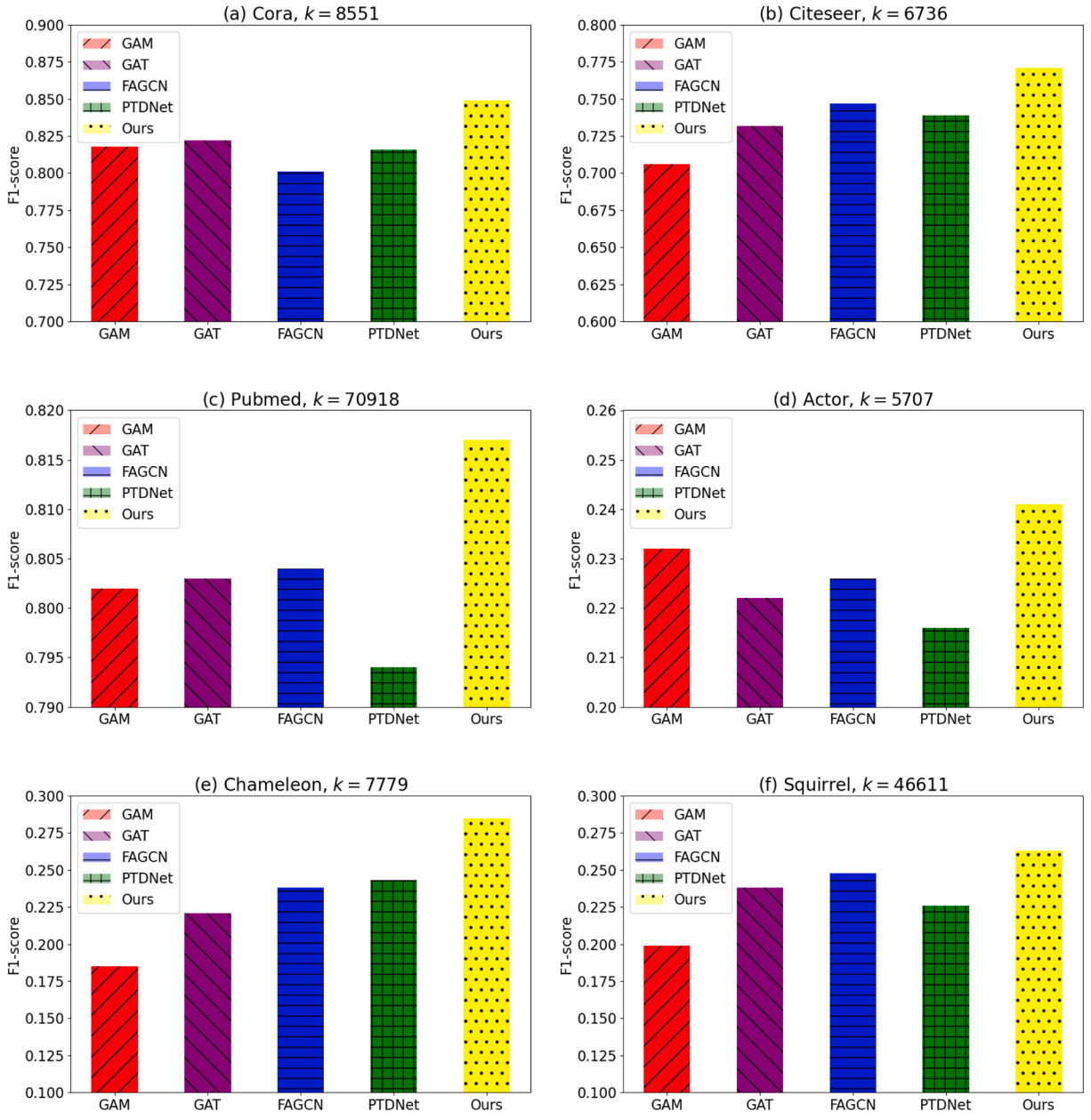


**Fig. 4.** (RQ2) F1-score evaluation for edge classification performance on six graph datasets. Our model is compared with four baselines that specify edge coefficients.

generated graphs from the final representations of GNNs, keeping the hyper-parameters the same as their *implementations*.[2] (5) Ours: The coefficients of our model are retrieved through Eq. (13).

In Fig. 4, we see that GAM shows the lowest performance for most datasets, except for (d) Actor, due to only utilizing a central node for prediction. GAT relatively outperforms GAM with the help of message passing and attention layers. Except for (a) Cora, FAGCN shows better performance than GAT, indicating the effectiveness of high-frequency signals. PTDNet is not as powerful, where edge pruning between communities fails to generalize on most datasets. Comparatively, our model significantly improves the F1 score for all datasets, justifying the necessity of confidence-aware subgraph matching.

[2] https://github.com/flyingdoog/PTDNet

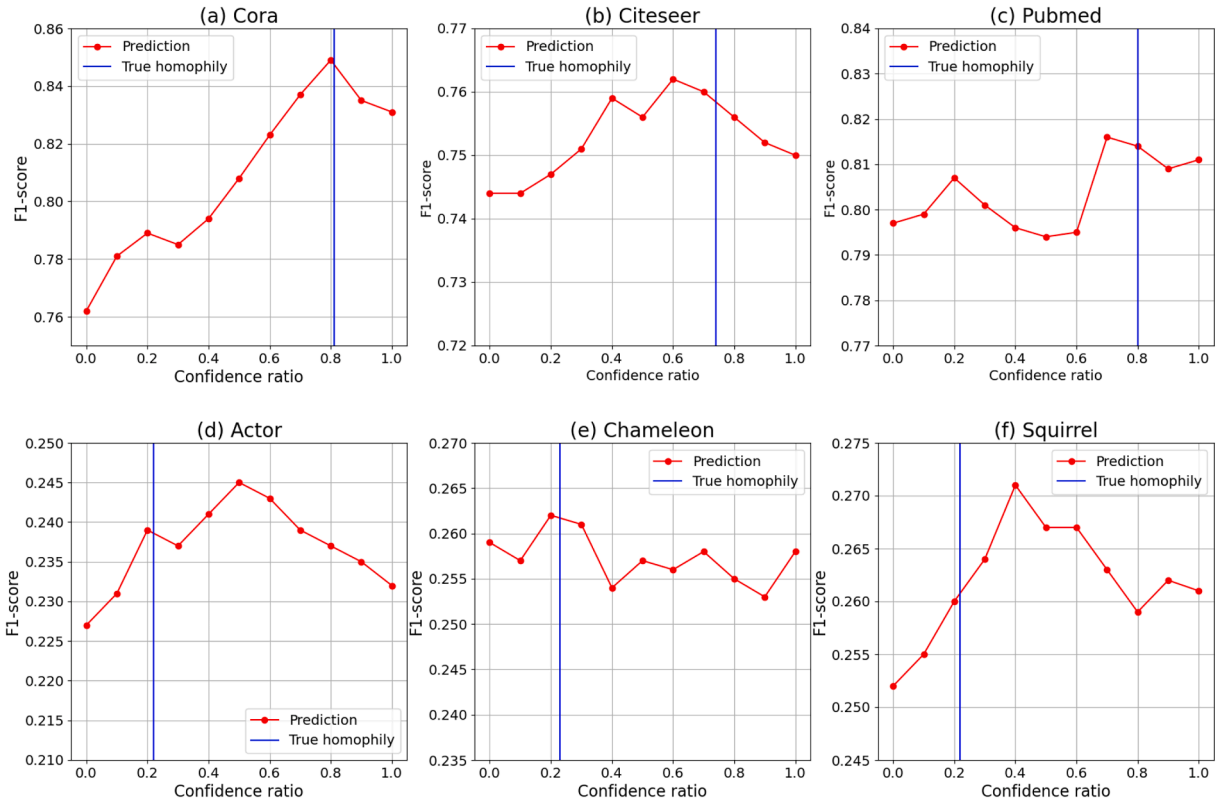


**Fig. 5.** (RQ3) F1-score variation with different confidence ratios of the subgraph matching module on six graph datasets.

**Table 5**
(RQ5) using two homophilic (Cora, Citeseer) and two heterophilic (Chameleon, Squirrel) graphs, we measure the Area Under Curve (AUC) score of the link prediction.

| Datasets | Cora | Citeseer | Chameleon | Squirrel |
|---|---|---|---|---|
| Hom. ratio ($h$) | 0.81 | 0.74 | 0.23 | 0.22 |
| VAGE [66] | 90.9 | 89.8 | 81.4 | 73.3 |
| GCN [8] | 91.2 | 90.3 | 83.5 | 74.2 |
| DropEdge (GCN) [63] | **91.5*** | **90.6*** | 84.0 | 76.1 |
| GAT [10] | 91.1 | 90.0 | 81.9 | 66.3 |
| GIN [58] | 87.1 | 86.1 | 78.7 | 62.7 |
| $H_2$GCN [64] | 86.7 | 84.7 | 79.2 | 73.0 |
| ConSM (Ours) | 91.3 | 90.5 | **84.1*** | **76.5*** |

### 6.5. Parameter sensitivity analysis (RQ3)

In this section, we further measure edge classification scores by varying a hyper-parameter of the subgraph matching module. To deal with heterophily, we introduced a confidence ratio ($\zeta$) to reflect data homophily, assuming that connected nodes may not share the same labels. In Fig. 5, we plot F1-scores on six datasets by varying $\zeta$ from 0 to 1. We also describe the true homophily ratio (please refer to $h$ in Tables 3 and 4) as blue lines. If $\zeta = 0$, the supplementary module does not utilize neighboring nodes for prediction, while $\zeta = 1$ means that it fully utilizes adjacent nodes.

Here, a confidence ratio ($\zeta$) that shows the best F1-score fairly aligns well with the true homophily ratio $h$, indicating the importance of selecting $\zeta$ for precise prediction. Though $\zeta = 0.5$ is quite different from $h = 0.22$ for (d) Actor, we assert that disassortative neighbors can also contribute to improving classifications, as described in Fig. 1. Nonetheless, we acknowledge that choosing $\zeta$ is quite sensitive and may require human efforts to achieve the best accuracy. This highlights the necessity of an automated or adaptive method for determining the optimal confidence ratio in future work.

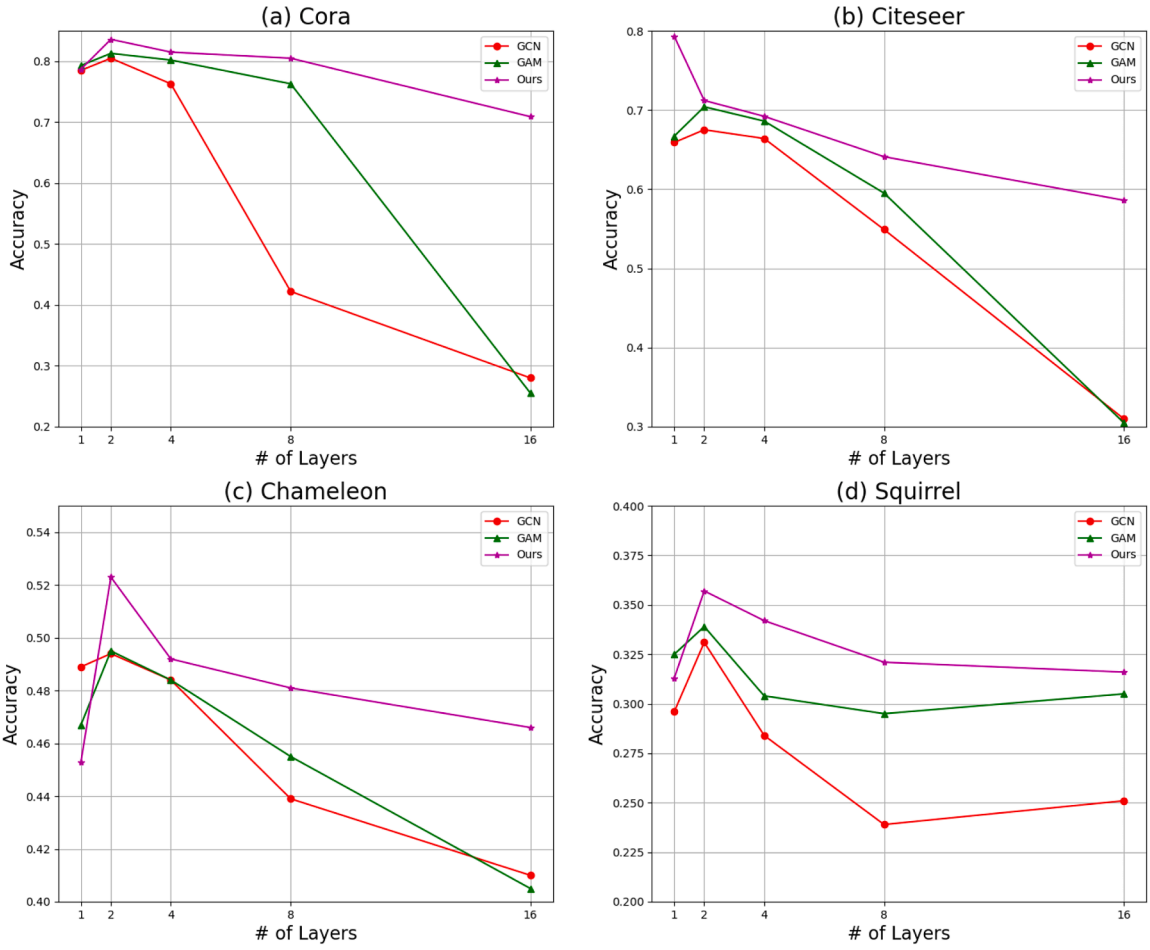


**Fig. 6.** (RQ4) Evaluation on over-smoothing using (a) Cora, (b) Citeseer, (c) Chameleon, and (d) Squirrel dataset. We plot the accuracy of two baselines and our method using different numbers of layers.

### 6.6. *Analysis on over-smoothing (RQ4)*

Over-smoothing is a fundamental problem for GNNs when stacking multiple layers [54,67]. Here, we scrutinize this phenomenon by varying the depth of layers as {1, 2, 4, 8, 16}, and report the node classification accuracy on (a) Cora, and (b) Chameleon. In Fig. 6, we describe the results of GCN, GAM, and our ConSM. GCN shows the best performance at 2 layers on both datasets. However, performance degrades slightly at 4 layers and dramatically decreases beyond that, indicating that GCN alone cannot alleviate the over-smoothing problem. Though GAM remains relatively stable compared to GCN, it also suffers from smoothing when stacking more layers. Comparatively, ConSM consistently achieves the best performance, and the accuracy does not decrease severely even for deeper layers (e.g., 16 layers).

We suggest that the integration of well-classified edge coefficients with label propagation effectively mitigates this problem, demonstrating the effectiveness of our method. This ability to maintain high performance across multiple layers underscores the robustness of ConSM in dealing with deeper architectures, which is critical for complex graph datasets.

### 6.7. *Link prediction (RQ5)*

To further demonstrate the effectiveness of our method, we conduct an additional experiment regarding its capability for link prediction. In detail, we randomly removed 15 % of edges and allocated 5 % as a validation set and 10 % as a test set, following the methodology in [63,66]. More specifically, edge weights are calculated through the multiplication of feature vectors of the connected nodes. Table 5 shows the Area Under the Curve (AUC) for state-of-the-art methods and our method. As demonstrated, DropEdge [63] slightly outperforms our method on homophilic graphs (Cora and Citeseer). However, as homophily decreases, ConSM attains the best performance on heterophilic graphs (Chameleon and Squirrel), which proves the efficacy of our method under heterophily. This additional experiment underscores the versatility of ConSM in not only node classification but also in link prediction tasks, further validating its robustness across different types of graph data (Table 6).

**Table 6**
(RQ6) We evaluate the node classification accuracy (%) by varying the number of training samples (L/C). The original semi-supervised learning uses 20 nodes per class.

| Dataset | Cora | | | Actor | | | Chameleon | | |
|---|---|---|---|---|---|---|---|---|---|
| L/C | 20 | 40 | 80 | 20 | 40 | 80 | 20 | 40 | 80 |
| GCN [8] | 79.1 | 82.8 | 83.4 | 20.4 | 20.7 | 21.1 | 49.4 | 53.5 | 55.7 |
| GAT [10] | 80.1 | 82.4 | 83.0 | 22.5 | 23.1 | 23.6 | 46.9 | 52.4 | 54.1 |
| APPNP [59] | 81.2 | 83.0 | 83.9 | 21.5 | 22.1 | 22.9 | 45.0 | 50.7 | 52.2 |
| ConSM (Ours) | **83.6*** | **84.5*** | **84.8*** | **27.7*** | **29.0*** | **29.2*** | **52.3*** | **53.7*** | **56.0*** |

**Table 7**
Statistical details of heterophilic datasets. For Chameleon and Squirrel, values outside parentheses correspond to the filtered datasets, while values in (parentheses) denote the original datasets.

| Datasets | Chameleon-filtered | Squirrel-filtered |
|---|---|---|
| # Nodes | 890 (2,277) | 2223 (5,201) |
| # Edges† | 8854 (33,824) | 46,998 (211,872) |
| # Features | 2325 (2,325) | 2089 (2,089) |
| # Classes | 5 (5) | 5 (5) |
| # Training Nodes | 100 (100) | 100 (100) |
| # Validation Nodes | 395 (1,088) | 1061 (2,550) |
| # Test Nodes | 395 (1,089) | 1062 (2,551) |

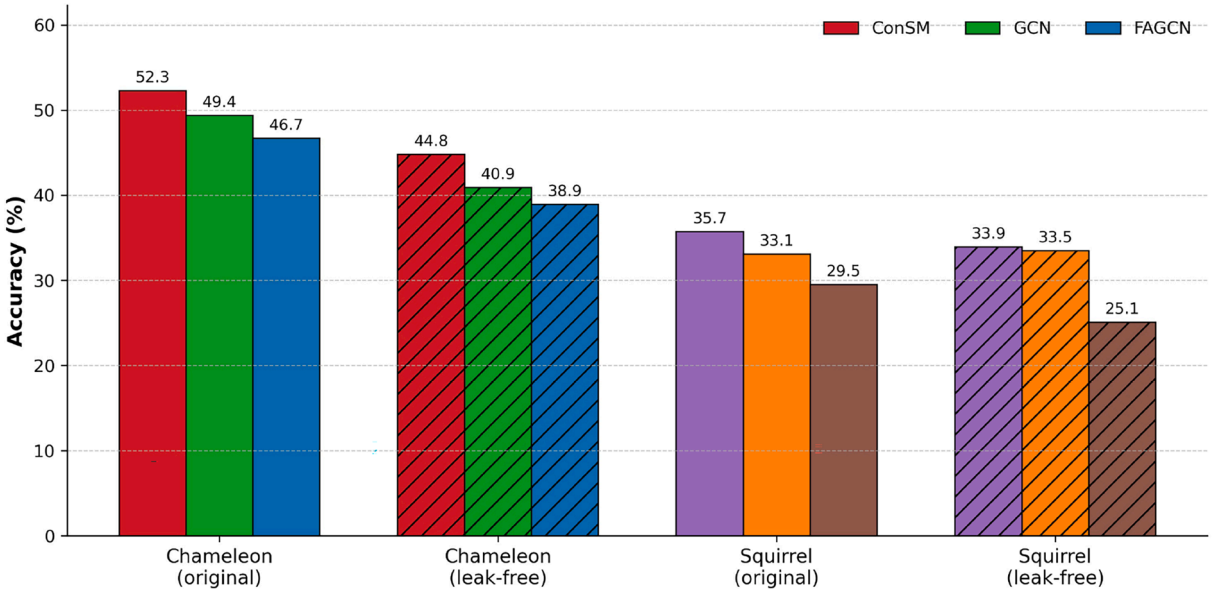


**Fig. 7.** Impact of data leakage removal on heterophilic benchmarks (Chameleon and Squirrel) using two benchmarks (GCN [8], FAGCN [14]) and our ConSM.

### 6.8. Impact of data sparsity (RQ6)

For a fair comparison, we evaluate one homophilic network (Cora) and two heterophilic graphs (Actor and Chameleon) in Table 6. We also employ GCN [8], GAT [10], and APPNP [59] as baselines. The results indicate that the performance of all methods increases proportionally with L/C. Notably, our method demonstrates a consistently higher accuracy across all datasets. Although the rate of improvement diminishes as L/C increases, our method maintains its lead, highlighting its robustness and effectiveness in different graph structures. This consistent performance underscores the superiority of our approach in leveraging the training data to enhance node classification accuracy.

### 6.9. Performance on leakage-free splits (RQ7)

To address concerns about train-test leakage in heterophilic benchmarks, we further evaluate ConSM on the leakage-free versions of Chameleon and Squirrel introduced by [48], where duplicated nodes are removed before constructing the data splits. As summarized in Table 7, the filtered datasets are substantially smaller than the original ones, indicating that the original benchmarks may overestimate model performance. Fig. 7 shows that all methods experience a noticeable drop in accuracy under the leakage-free setting, confirming that the original splits contain shortcut signals introduced by duplicated nodes. Nevertheless, ConSM remains consistently strong across both the original and filtered versions of Chameleon and Squirrel, and it continues to outperform or closely match competitive baselines after leakage removal. This result suggests that the benefit of ConSM does not stem from benchmark artifacts, but from its ability to identify structurally informative neighbors even in a stricter evaluation setting.

## 7. Conclusion

In this work, we propose a confidence ratio to handle multiple disassortative edges for semi-supervised node classification. We highlighted the significance of precisely configuring edge weights and thus proposed measuring similarity between two connected nodes using their subgraphs. Further, based on observations that directly applying the predicted weights can be highly risky, we integrate label propagation with our confidence ratio to secure robustness and improve overall performance. Extensive experiments on both homophilic and heterophilic setups demonstrate the superiority of our model. Our findings indicate that the proposed ConSM model significantly enhances classification accuracy by effectively managing the inherent complexities in graph data. The method's robustness against over-smoothing and its adaptability through the confidence ratio make it a versatile solution for various graph-based learning tasks. Future work will explore automated ways to determine the optimal confidence ratio and extend the model's applicability to other types of graph structures and real-world applications.

## CRediT authorship contribution statement

**Yoonhyuk Choi:** Writing – review & editing, Writing – original draft, Visualization, Validation, Supervision, Software, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization; **Chong-Kwon Kim:** Writing – review & editing, Supervision, Project administration, Funding acquisition, Formal analysis.

## Data availability

Data will be made available on request.

## Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Chong-Kwon Kim reports financial support was provided by Seoul National University. First author is working as an Assistant Professor at Sookmyung Women's University. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

## Acknowledgment

This work was supported by the KENTECH Research Grant (202200019A), by the National Research Foundation of Korea (NRF) grant funded by the Korea government (MSIT) (RS-2026-25468807, RS-2026-2548877), and by the Institute of Information & communications Technology Planning & Evaluation(IITP) under the Leading Generative AI Human Resources Development (IITP-2026-25544527) grant funded by the Korea government (MSIT).

## References

[1] Y. Choi, J. Choi, T. Ko, H. Byun, C.-K. Kim, Finding heterophilic neighbors via confidence-based subgraph matching for semi-supervised node classification, in: Proceedings of the 31st ACM International Conference on Information & Knowledge Management, 2022, pp. 283–292.
[2] J. Gilmer, S.S. Schoenholz, P.F. Riley, O. Vinyals, G.E. Dahl, Neural message passing for quantum chemistry, in: International Conference on Machine Learning, PMLR, 2017, pp. 1263–1272.
[3] A. Fout, J. Byrd, B. Shariat, A. Ben-Hur, Protein interface prediction using graph convolutional networks, Adv. Neural Inf. Process. Syst. 30 (2017), pp. 6530–6539.
[4] W. Fan, Y. Ma, Q. Li, Y. He, E. Zhao, J. Tang, D. Yin, Graph neural networks for social recommendation, in: The World Wide Web Conference, 2019, pp. 417–426.
[5] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, Nature 521 (7553) (2015) 436–444.
[6] F. Scarselli, M. Gori, A.C. Tsoi, M. Hagenbuchner, G. Monfardini, The graph neural network model, IEEE Trans. Neural Netw. 20 (1) (2008) 61–80.
[7] M. Defferrard, X. Bresson, P. Vandergheynst, Convolutional neural networks on graphs with fast localized spectral filtering, Adv. Neural Inf. Process. Syst. 29 (2016).
[8] T.N. Kipf, M. Welling, Semi-supervised classification with graph convolutional networks, (2016). arXiv:1609.02907
[9] W. Hamilton, Z. Ying, J. Leskovec, Inductive representation learning on large graphs, Adv. Neural Inf. Process. Syst. 30 (2017).
[10] P. Velickovic, G. Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, Graph attention networks, Stat 1050 (2017) 20.
[11] M. McPherson, L. Smith-Lovin, J.M. Cook, Birds of a feather: homophily in social networks, Annu. Rev. Sociol. 27 (1) (2001) 415–444.

[12] S. Pandit, D.H. Chau, S. Wang, C. Faloutsos, Netprobe: a fast and scalable system for fraud detection in online auction networks, in: Proceedings of the 16th International Conference on World Wide Web, 2007, pp. 201–210.
[13] L. Yang, F. Wu, Y. Wang, J. Gu, Y. Guo, Masked graph convolutional network, in: IJCAI, 2019, pp. 4070–4077.
[14] D. Bo, X. Wang, C. Shi, H. Shen, Beyond low-frequency information in graph convolutional networks, (2021). arXiv:2101.00797
[15] D. Kim, A. Oh, How to find your friendly neighborhood: graph attention design with self-supervision, (2022). arXiv:2204.04879
[16] Z. Ying, D. Bourgeois, J. You, M. Zitnik, J. Leskovec, Gnnexplainer: generating explanations for graph neural networks, Adv. Neural Inf. Process. Syst. 32 (2019) 9244–9255.
[17] N. Entezari, S.A. Al-Sayouri, A. Darvishzadeh, E.E. Papalexakis, All you need is low (rank) defending against adversarial attacks on graphs, in: Proceedings of the 13th International Conference on Web Search and Data Mining, 2020, pp. 169–177.
[18] D. Luo, W. Cheng, W. Yu, B. Zong, J. Ni, H. Chen, X. Zhang, Learning to drop: robust graph neural network via topological denoising, in: Proceedings of the 14th ACM International Conference on Web Search and Data Mining, 2021, pp. 779–787.
[19] H. Pei, B. Wei, K.C.-C. Chang, Y. Lei, B. Yang, Geom-GCN: geometric graph convolutional networks, (2020). arXiv:2002.05287
[20] T. Yang, Y. Wang, Z. Yue, Y. Yang, Y. Tong, J. Bai, Graph pointer neural networks, (2021). arXiv:2110.00973
[21] W. Jin, T. Derr, Y. Wang, Y. Ma, Z. Liu, J. Tang, Node similarity preserving graph convolutional networks, in: Proceedings of the 14th ACM International Conference on Web Search and Data Mining, 2021, pp. 148–156.
[22] T. Xiao, Z. Chen, D. Wang, S. Wang, Learning how to propagate messages in graph neural networks, in: Proceedings of the 27th ACM SIGKDD Conference on Knowledge Discovery & Data Mining, 2021, pp. 1894–1903.
[23] A.K. McCallum, K. Nigam, J. Rennie, K. Seymore, Automating the construction of internet portals with machine learning, Inf. Retr. 3 (2) (2000) 127–163.
[24] B. Rozemberczki, R. Davies, R. Sarkar, C. Sutton, Gemsec: graph embedding with self clustering, in: Proceedings of the 2019 IEEE/ACM International Conference on Advances in Social Networks Analysis and Mining, 2019, pp. 65–72.
[25] S. Luan, C. Hua, Q. Lu, J. Zhu, M. Zhao, S. Zhang, X.-W. Chang, D. Precup, Is heterophily a real nightmare for graph neural networks to do node classification? (2021). arXiv:2109.05641
[26] Y. Ma, X. Liu, N. Shah, J. Tang, Is homophily a necessity for graph neural networks?, (2021). arXiv:2106.06134
[27] O. Stretcu, K. Viswanathan, D. Movshovitz-Attias, E. Platanios, S. Ravi, A. Tomkins, Graph agreement models for semi-supervised learning, Adv. Neural Inf. Process. Syst. 32 (2019) 8713–8723.
[28] G. Peyré, M. Cuturi, et al., Computational optimal transport: with applications to data science, Found. Trends/textregistered Mach. Learn. 11 (5-6) (2019) 355–607.
[29] H. Xu, D. Luo, H. Zha, L.C. Duke, Gromov-wasserstein learning for graph matching and node embedding, in: International Conference on Machine Learning, PMLR, 2019, pp. 6932–6941.
[30] G. Mialon, D. Chen, A. d'Aspremont, J. Mairal, A trainable optimal transport embedding for feature aggregation and its relationship to attention, (2020). arXiv:2006.12065
[31] S. Kolouri, N. Naderializadeh, G.K. Rohde, H. Hoffmann, Wasserstein embedding for graph learning, (2020). arXiv:2006.09430
[32] T.D. Bui, S. Ravi, V. Ramavajjala, Neural graph learning: training neural networks using graphs, in: Proceedings of the Eleventh ACM International Conference on Web Search and Data Mining, 2018, pp. 64–71.
[33] J. Zhu, Y. Yan, M. Heimann, L. Zhao, L. Akoglu, D. Koutra, Heterophily and graph neural networks: past, present and future, IEEE Data Eng. Bull. (2023) https://par.nsf.gov/biblio/10435541.
[34] S. Luan, C. Hua, Q. Lu, L. Ma, L. Wu, X. Wang, M. Xu, X.-W. Chang, D. Precup, R. Ying, et al., The heterophilic graph learning handbook: benchmarks, models, theoretical analysis, applications and challenges, (2024). arXiv:2407.09618
[35] D.K. Hammond, P. Vandergheynst, R. Gribonval, Wavelets on graphs via spectral graph theory, Appl. Comput. Harmon. Anal. 30 (2) (2011) 129–150.
[36] Y. Dong, K. Ding, B. Jalaian, S. Ji, J. Li, Graph neural networks with adaptive frequency response filter, (2021). arXiv:2104.12840
[37] X. Li, R. Zhu, Y. Cheng, C. Shan, S. Luo, D. Li, W. Qian, Finding global homophily in graph neural networks when meeting heterophily, in: International Conference on Machine Learning, PMLR, 2022, pp. 13242–13256.
[38] Y. Choi, T. Ko, J. Choi, C.-K. Kim, Beyond binary: improving signed message passing in graph neural networks for multi-class graphs, IEEE Trans. Pattern Anal. Mach. Intell. (2025) 47 9535–9549.
[39] Y. Chen, Y. Luo, J. Tang, L. Yang, S. Qiu, C. Wang, X. Cao, LSGNN: towards general graph neural network in node classification by local similarity, (2023). arXiv:2305.04225
[40] L. Liang, X. Hu, Z. Xu, Z. Song, I. King, Predicting global label relationship matrix for graph neural networks under heterophily, Adv. Neural Inf. Process. Syst. 36 (2023) 10909–10921.
[41] C. Zheng, B. Zong, W. Cheng, D. Song, J. Ni, W. Yu, H. Chen, W. Wang, Robust graph representation learning via neural sparsification, in: International Conference on Machine Learning, PMLR, 2020, pp. 11458–11468.
[42] U. Desai, S. Bandyopadhyay, S. Tamilselvam, Graph neural network to dilute outliers for refactoring monolith application, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, 2021, pp. 72–80.
[43] X. Wang, R. Girshick, A. Gupta, K. He, Non-local neural networks, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2018, pp. 7794–7803.
[44] M. Liu, Z. Wang, S. Ji, Non-local graph neural networks, IEEE Trans. Pattern Anal. Mach. Intell. 44 (2021) 10270–10276.
[45] M. Zou, Z. Gan, R. Cao, C. Guan, S. Leng, Similarity-navigated graph neural networks for node classification, Inf. Sci. 633 (2023) 41–69.
[46] Y. Liu, Y. Zheng, D. Zhang, V.C.S. Lee, S. Pan, Beyond smoothing: unsupervised graph representation learning with edge heterophily discriminating, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 37, 2023, pp. 4516–4524.
[47] H. Liu, B. Hu, X. Wang, C. Shi, Z. Zhang, J. Zhou, Confidence may cheat: self-training on graph neural networks under distribution shift, in: Proceedings of the ACM Web Conference 2022, 2022, pp. 1248–1258.
[48] O. Platonov, D. Kuznedelev, M. Diskin, A. Babenko, L. Prokhorenkova, A critical look at the evaluation of GNNs under heterophily: are we really making progress?, (2023). arXiv:2302.11640
[49] O. Platonov, D. Kuznedelev, A. Babenko, L. Prokhorenkova, Characterizing graph datasets for node classification: homophily-heterophily dichotomy and beyond, Adv. Neural Inf. Process. Syst. 36 (2023) 523–548.
[50] J. Wang, Y. Guo, L. Yang, Y. Wang, Understanding heterophily for graph neural networks, (2024). arXiv:2401.09125
[51] H. Yang, K. Ma, J. Cheng, Rethinking graph regularization for graph neural networks, in: Proceedings of the AAAI Conference on Artificial Intelligence, Vol. 35, 2021, pp. 4573–4581.
[52] H. Xu, D. Luo, L. Carin, Scalable gromov-wasserstein learning for graph partitioning and matching, Adv. Neural Inf. Process. Syst. 32 (2019) 3052–3062.
[53] Y. Choi, J. Choi, C.-K. Kim, Sheaf graph neural networks via PAC-bayes spectral optimization, (2025). arXiv:2508.00357
[54] L. Zhao, L. Akoglu, Pairnorm: tackling oversmoothing in GNNs, (2019). arXiv:1909.12223
[55] M. Cuturi, A. Doucet, Fast computation of Wasserstein barycenters, in: International Conference on Machine Learning, PMLR, 2014, pp. 685–693.
[56] R. Sinkhorn, P. Knopp, Concerning nonnegative matrices and doubly stochastic matrices, Pac. J. Math. 21 (2) (1967) 343–348.
[57] H. Wang, J. Leskovec, Unifying graph convolutional neural networks and label propagation, (2020). arXiv:2002.06755
[58] K. Xu, W. Hu, J. Leskovec, S. Jegelka, How powerful are graph neural networks?, (2018). arXiv:1810.00826
[59] J. Klicpera, A. Bojchevski, S. Günnemann, Predict then propagate: graph neural networks meet personalized pagerank, (2018). arXiv:1810.05997
[60] M. Chen, Z. Wei, Z. Huang, B. Ding, Y. Li, Simple and deep graph convolutional networks, in: International Conference on Machine Learning, PMLR, 2020, pp. 1725–1735.
[61] J. Tang, J. Sun, C. Wang, Z. Yang, Social influence analysis in large-scale networks, in: Proceedings of the 15th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2009, pp. 807–816.

[62] M.-C. Popescu, V.E. Balas, L. Perescu-Popescu, N. Mastorakis, Multilayer perceptron and neural networks, WSEAS Trans. Circuits Syst. 8 (7) (2009) 579–588.
[63] Y. Rong, W. Huang, T. Xu, J. Huang, DropEdge: towards deep graph convolutional networks on node classification, (2019). arXiv:1907.10903
[64] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, D. Koutra, Beyond homophily in graph neural networks: current limitations and effective designs, Adv. Neural Inf. Process. Syst. 33 (2020) 7793–7804.
[65] G. Luo, J. Li, J. Su, H. Peng, C. Yang, L. Sun, P.S. Yu, L. He, Graph entropy guided node embedding dimension selection for graph neural networks, (2021). arXiv:2105.03178
[66] T.N. Kipf, M. Welling, Variational graph auto-encoders, (2016). arXiv:1611.07308
[67] Q. Li, Z. Han, X.-M. Wu, Deeper insights into graph convolutional networks for semi-supervised learning, in: Thirty-Second AAAI Conference on Artificial Intelligence, 2018, pp. 0–0.